



Development Ticket: TKT-9055

Feature: Enterprise Order Logistics System & N-Tier Architecture

Context

We are building the backend for a global e-commerce platform. The system is divided into strict architectural layers: Data Beans (to hold secure information) and Business Services (to process that information). Your task is to build this system using strict Object-Oriented principles across multiple packages.

📦 Package 1: com.zen.bean (Data Layer)

This package handles Encapsulation and Inheritance.

1. The Order Class (Base)

- Create a `public` class named `Order`.
- Define two `private` variables: `orderId` (String) and `orderAmount` (double).
- Create getters and setters with strict business rules:
 - **Rule for setOrderId:** All internal orders must start with the prefix "ORD-". If the passed string does not start with "ORD-", print a warning and set the value to "ORD-UNKNOWN".
 - **Rule for setOrderAmount:** An order amount cannot be free or negative. If the value is ≤ 0 , print a warning and set the amount to 1.0.

2. The OverseasOrder Class (Child)

- Create a `public` class named `OverseasOrder` that extends the `Order` class.
- Define a new `private` variable: `countryCode` (String).
- **Rule for setCountryCode:** The code must be exactly 3 characters long (e.g., "USA", "IND"). If it is not exactly 3 characters, print a warning and set it to "XXX".

📦 Package 2: com.zen.service (Business Logic Layer)

This package handles Abstraction, Method Overriding, and Method Overloading.

1. The LogisticsService Class (Abstraction)

- Create a `public abstract` class named `LogisticsService`.
- **Abstraction:** Define an abstract method:
`public abstract double calculateShipping(Order order);` (Note: You will need to import the `Order` class).
- **Overloading (Variation 1):** Create a concrete method named `generateManifest(Order order)`. It should print out the `orderId` and `orderAmount` using getters.
- **Overloading (Variation 2):** Create a second concrete method:
`generateManifest(OverseasOrder order, String carrierName)`. This must print the `orderId`, `orderAmount`, `countryCode`, AND the `carrierName` to ensure specialized data is utilized.

2. The StandardLogistics & ExpressLogistics Classes (Polymorphism)

- Create a class named `StandardLogistics` that extends `LogisticsService`. **Override** the `calculateShipping` method to return exactly **5%** of the order amount.
- Create a class named `ExpressLogistics` that extends `LogisticsService`. **Override** the `calculateShipping` method to return exactly **15%** of the order amount.



Package 3: com.zen (Execution Layer)

Create a `Main` class with your main method to test the architecture:

1. Instantiate one `Order` and one `OverseasOrder`.
2. Intentionally pass **invalid data** into their setters (missing "ORD-" prefix, negative amount, 2-character country code) to prove encapsulation validation works.
3. Instantiate a `StandardLogistics` service and an `ExpressLogistics` service.
4. Pass the base `Order` into the standard service to calculate the shipping fee, and call the first version of `generateManifest`.
5. Pass the `OverseasOrder` into the express service to calculate the shipping fee, and call the second version of `generateManifest` (with a carrier name) to prove the overloaded method works.